

Pseudo-código

Aprende como escribir algoritmos de manera estructurada y cercana a código real de una aplicación.

Qué es

Pseudo-código (pseudo = falso) se refiere a *cualquier* representación de un algoritmo.

Existe pseudo-código no gráfico (como el que hemos visto) y gráfico (diagramas de flujo).

Algoritmos de uso real

Hasta ahora sólo hemos visto algoritmos basados en acciones de la vida real.

Sin embargo, los algoritmos de la programación no son tan distintos como podrías pensar.

La diferencia principal es que siguen una estructura más rígida y se ordenan de distintas maneras.

Por ejemplo, analicemos este código de Javascript (no tienes que entenderlo, solo quiero demostrar que por debajo, no es más que un algoritmo como los que hemos visto):

```
1 function onClick()
2   const numberLabelElement = document.getElementById('number-label')
3   const currentNumberText = numberLabelElement.innerHTML
4   const currentNumber = Number(currentNumberText)
5   const newNumber = currentNumber + 1
6   numberLabelElement.innerHTML = currentNumberText + ' ' + newNumber
7 }
```

[Copiar](#)

Podemos observar que Javascript, al ser un lenguaje de programación, tiene cierta **sintaxis** específica que se debe seguir.

Sin embargo, cada línea es una instrucción, justo como las que hemos visto.

Si quisiéramos pasar este código a pseudo-código, se vería algo así:

Hacer que botón sume click

1. Inicio (Cada vez que se haga click en el botón):
2. Buscar el texto que muestra el numero de clicks.
3. Guardar su valor.
4. Convertir su valor a numero.
5. Guardar el valor de la suma del numero + 1.
6. Cambiar al valor del texto a este numero numero.
7. Fin

```
1 function onClick()
2   const numberLabelElement = document.getElementById('numberLabel')
3   const currentNumberText = numberLabelElement.innerHTML
4   const currentNumber = Number(currentNumberText)
5   const newNumber = currentNumber + 1
6   numberLabelElement.innerHTML = newNumber
7 }
```

Copiar

Especificación de pseudo-código

Con el fin de acercarnos más a código real como este, vamos a ver algoritmos usando una especificación de pseudocódigo no gráfico más estricta.

Vamos a escribir algoritmos utilizando solo las siguientes instrucciones:

Palabra	Uso
Inicio	Comienzo del algoritmo
Fin	Final del algoritmo
Definir <nombre>	Definir una variable
Mostrar <texto>	Mostrar algo en pantalla
Pedir <nombre>	Pedirle al usuario que ingrese un valor para una variable
Si (<condición>) entonces: <instrucciones> Sino <instrucciones alternativas> FinSi	Realizar acciones solo si una condición se cumple, o realizar otra.
Mientras que (<condición>) repetir: <instrucciones> FinMientras	Realizar acciones de manera repetida hasta que una condición se deje de cumplir

Para las condiciones, vamos a usar solo las siguientes comparaciones:

Operación	Símbolo	Descripción	Ejemplos
Igual a	==	Revisa si dos valores son iguales	$10 == 10 \rightarrow \text{Verdadero}$ $20 == 20 \rightarrow \text{Verdadero}$ $10 == 20 \rightarrow \text{Falso}$
Distinto a	!=	Revisa si dos valores son distintos	$10 != 10 \rightarrow \text{Falso}$ $20 != 20 \rightarrow \text{Falso}$ $10 != 20 \rightarrow \text{Verdadero}$
Mayor a	>	Revisa si un numero es mayor a otro	$90 > 40 \rightarrow \text{Verdadero}$ $40 > 90 \rightarrow \text{Falso}$ $50 > 50 \rightarrow \text{Falso}$
Mayor o igual a	>=	Revisa si un numero es mayor o igual a otro	$90 >= 40 \rightarrow \text{Verdadero}$ $40 >= 90 \rightarrow \text{Falso}$ $50 >= 50 \rightarrow \text{Verdadero}$
Menor a	<	Revisa si un numero es menor a otro	$90 < 40 \rightarrow \text{Falso}$ $40 < 90 \rightarrow \text{Verdadero}$ $50 < 50 \rightarrow \text{Falso}$
Menor o igual a	<=	Revisa si un numero es menor o igual a otro	$90 <= 40 \rightarrow \text{Falso}$ $40 <= 90 \rightarrow \text{Verdadero}$ $50 <= 50 \rightarrow \text{Verdadero}$

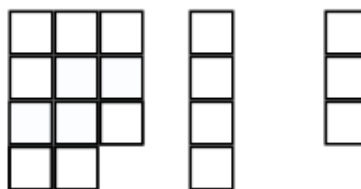
Tambien podemos hacer operaciones aritméticas:

Operación	Símbolo	Descripción	Ejemplos
Suma	+	Suma dos números	$10 + 35 \rightarrow 45$
Resta	-	Resta un numero de otro	$90 - 50 \rightarrow 40$
Multiplicación	*	Multiplica dos números	$3 * 5 \rightarrow 15$
Division	/	Divide un numero por otro	$10 / 2 \rightarrow 5$
Potencia	**	Eleva un numero a la potencia de otro	$2 ** 3 \rightarrow 8 (2 * 2 * 2)$

Operación	Símbolo	Descripción	Ejemplos
Mod (Modulo)	%	Divide un numero por otro pero retorna su residuo (explicación más a fondo abajo)	10 % 3 → 1
Division entera	//	Divide un numero por otro pero quitando los decimales del resultado	10 // 3 → 3 (3.33)

Modulo

Modulo operation



$$11 \text{ mod } 4 = 3$$

ComputerHope.com

Variables

Nos permiten guardar datos a los que podemos usar en futuras instrucciones del algoritmo.

Por así decirlo, es como apuntar algo en una nota.

O como meter un objeto en una caja.

Todas las variables tiene un `identificador` (su nombre).

Su nombre solo puede contener letras, números o guiones bajos.

Tambien nos permiten hacer comparaciones y operaciones matemáticas.

Por ejemplo, supongamos que tenemos un algoritmo en el que el que le pedimos su `edad` al usuario, y le mostramos si es mayor de edad o no:

En la línea 2 definimos que tendremos una variable llamada `edad`.

El la línea 4, le pedimos al usuario que la ingrese

En la línea 5 , revisamos si la edad ingresada es mayor o igual a 18

Veamos otro ejemplo:

A un trabajador le descuentan 10% si su sueldo es menor o igual a 1000.

Por encima de 1000 y debajo de 2000, le descuentan 5% del adicional.

Por encima de 2000, el 3% del adicional.

Vamos a hacer un algoritmo que calcule el descuento y el sueldo neto que recibe el trabajador.